

General Tech Companies Interview Preparation

Airbnb | Stripe | DoorDash | Snowflake | Databricks | LinkedIn | Atlassian

AIRBNB

Company Profile

- **Mission:** Belong anywhere — connecting hosts and guests globally
- Culture: Belonging, championing the community, data-driven product decisions
- Engineering: Python-heavy stack, Airflow (originated here), Superset (originated here)
- DB Stack: MySQL (primary OLTP), Apache Airflow + Hive + Presto (analytics), Druid (real-time), Elasticsearch (search), Redis (cache)
- PostgreSQL: Used for internal tools and finance

Interview Process

Round	Format	Duration	Focus
Technical Screen	CoderPad	60 min	SQL + Python
Onsite 1	CoderPad	60 min	Complex SQL
Onsite 2	Whiteboard	60 min	Data modeling
Onsite 3	Video	60 min	System design
Onsite 4	Video	45 min	Behavioral (Core Values)

SQL Questions — Airbnb-Specific

Q1. Find hosts with listings in multiple cities.

```
SELECT host_id, COUNT(DISTINCT city) AS city_count, ARRAY_AGG(DISTINCT city) AS cities
FROM listings
GROUP BY host_id
HAVING COUNT(DISTINCT city) > 1
ORDER BY city_count DESC;
```

Q2. Calculate occupancy rate for listings over the last 90 days.

```
SELECT
  listing_id,
  COUNT(*) FILTER (WHERE booking_date IS NOT NULL) AS booked_nights,
  90 AS total_nights,
  ROUND(100.0 * COUNT(*) FILTER (WHERE booking_date IS NOT NULL) / 90, 2) AS occupancy_rate
FROM (
  SELECT
```

```

        l.listing_id,
        d.booking_date,
        b.booking_id
    FROM listings l
    CROSS JOIN generate_series(CURRENT_DATE - 89, CURRENT_DATE, '1 day'::interval)
    d(booking_date)
    LEFT JOIN bookings b ON l.listing_id = b.listing_id
                        AND d.booking_date BETWEEN b.checkin_date AND
b.checkout_date - 1
    ) occupancy_data
GROUP BY listing_id;

```

Q3. Find "super hosts" — top 10% by revenue and rating >= 4.8.

```

WITH host_metrics AS (
    SELECT
        host_id,
        SUM(total_price) AS total_revenue,
        AVG(guest_rating) AS avg_rating,
        COUNT(*) AS bookings,
        NTILE(10) OVER (ORDER BY SUM(total_price) DESC) AS revenue_decile
    FROM bookings b
    JOIN reviews r ON b.booking_id = r.booking_id
    WHERE b.status = 'completed'
        AND b.checkout_date >= NOW() - INTERVAL '365 days'
    GROUP BY host_id
)
SELECT host_id, total_revenue, avg_rating, bookings
FROM host_metrics
WHERE revenue_decile = 1 AND avg_rating >= 4.8
ORDER BY total_revenue DESC;

```

Q4. Calculate average booking lead time (days between booking and checkin).

```

SELECT
    city,
    ROUND(AVG(checkin_date - booked_at::date), 1) AS avg_lead_time_days,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY checkin_date - booked_at::date) AS
median_lead_time
FROM bookings b
JOIN listings l ON b.listing_id = l.listing_id
WHERE status = 'completed'
GROUP BY city
ORDER BY avg_lead_time_days;

```

Q5. Price elasticity: does lowering price increase bookings?

```

WITH monthly_metrics AS (
    SELECT
        listing_id,

```

```

        DATE_TRUNC('month', checkin_date) AS month,
        AVG(price_per_night) AS avg_price,
        COUNT(*) AS bookings
    FROM bookings
    WHERE status = 'completed'
    GROUP BY listing_id, DATE_TRUNC('month', checkin_date)
)
SELECT
    listing_id,
    CORR(avg_price, bookings) AS price_booking_correlation
FROM monthly_metrics
GROUP BY listing_id
HAVING COUNT(*) >= 6 -- at least 6 months of data
ORDER BY price_booking_correlation;

```

Database Design — Airbnb

```

CREATE TABLE listings (
    listing_id      BIGSERIAL PRIMARY KEY,
    host_id         BIGINT NOT NULL,
    property_type   VARCHAR(50),
    room_type       VARCHAR(30),
    city            VARCHAR(100),
    country         CHAR(2),
    location        GEOMETRY(POINT, 4326),
    price_per_night NUMERIC(10,2),
    min_nights      INT DEFAULT 1,
    amenities       TEXT[],
    is_instant_book BOOLEAN DEFAULT FALSE,
    avg_rating      NUMERIC(3,2),
    review_count    INT DEFAULT 0
);

CREATE TABLE availability_calendar (
    listing_id      BIGINT NOT NULL,
    calendar_date   DATE NOT NULL,
    is_available    BOOLEAN DEFAULT TRUE,
    price           NUMERIC(10,2), -- override price
    PRIMARY KEY (listing_id, calendar_date)
);

CREATE TABLE bookings (
    booking_id      BIGSERIAL PRIMARY KEY,
    listing_id      BIGINT NOT NULL,
    guest_id        BIGINT NOT NULL,
    checkin_date    DATE NOT NULL,
    checkout_date   DATE NOT NULL,
    total_price     NUMERIC(12,2),
    status          VARCHAR(20) DEFAULT 'requested',
    booked_at       TIMESTAMPTZ DEFAULT NOW(),
    EXCLUDE USING GIST (listing_id WITH =, DATERANGE(checkin_date, checkout_date)

```

```
WITH &&)
    -- Prevents double-bookings at DB level!
);
```

Salary Bands (USD)

Level	Base	RSU (4yr)	Total Comp
L3	\$140K-\$175K	\$80K-\$150K	\$180K-\$250K
L4	\$175K-\$215K	\$150K-\$280K	\$250K-\$370K
L5	\$215K-\$270K	\$280K-\$500K	\$370K-\$540K

STRIPE

Company Profile

- **Mission:** Increase the GDP of the internet — developer-first financial infrastructure
- Culture: Very high engineering bar, deep technical interviews, Unix philosophy ("do one thing well")
- Financial accuracy is religion — no rounding errors, no double charges, full auditability
- DB Stack: PostgreSQL (primary), Redis (cache), Hadoop/Spark (analytics), Kafka (events)
- Heavy PostgreSQL users — one of the most PostgreSQL-intensive FAANG-adjacent companies

Interview Process

Round	Format	Duration	Focus
Technical Screen	CoderPad	60 min	SQL + systems
Onsite 1	CoderPad	60 min	Deep SQL + financial logic
Onsite 2	Whiteboard	60 min	Database design (financial)
Onsite 3	Video	60 min	Distributed systems
Onsite 4	Video	45 min	Behavioral

SQL Questions — Stripe-Specific

Q1. Detect duplicate charges — same amount, same customer, within 60 seconds.

```
SELECT
    a.charge_id AS charge_a,
    b.charge_id AS charge_b,
    a.customer_id,
    a.amount,
    a.created_at AS time_a,
    b.created_at AS time_b
FROM charges a
JOIN charges b ON a.customer_id = b.customer_id
               AND a.amount = b.amount
               AND a.charge_id < b.charge_id
```

```

AND b.created_at BETWEEN a.created_at AND a.created_at + INTERVAL '60
seconds'

AND a.status = 'succeeded'
AND b.status = 'succeeded';

```

Q2. Calculate daily transaction volume with currency normalization (all to USD).

```

SELECT
    DATE(created_at) AS txn_date,
    COUNT(*) AS transaction_count,
    SUM(amount / 100.0 * fx.rate_to_usd) AS total_usd_volume
FROM charges c
JOIN fx_rates fx ON c.currency = fx.from_currency
                 AND DATE(c.created_at) = fx.rate_date
WHERE c.status = 'succeeded'
GROUP BY DATE(created_at)
ORDER BY txn_date DESC;

```

Q3. Payment flow analysis: calculate success rate by payment method and country.

```

SELECT
    payment_method,
    country,
    COUNT(*) AS attempts,
    COUNT(*) FILTER (WHERE status = 'succeeded') AS succeeded,
    ROUND(100.0 * COUNT(*) FILTER (WHERE status = 'succeeded') / COUNT(*), 2) AS
success_rate
FROM charges
WHERE created_at >= NOW() - INTERVAL '30 days'
GROUP BY payment_method, country
ORDER BY attempts DESC;

```

Q4. Find merchants with abnormally high refund rates (potential fraud).

```

WITH merchant_stats AS (
    SELECT
        merchant_id,
        COUNT(*) AS total_charges,
        SUM(amount) AS total_charged,
        COUNT(*) FILTER (WHERE refunded = TRUE) AS refunds,
        SUM(CASE WHEN refunded THEN amount END) AS refund_amount
    FROM charges
    WHERE created_at >= NOW() - INTERVAL '30 days'
    GROUP BY merchant_id
)
SELECT
    merchant_id,
    total_charges,
    refunds,
    ROUND(100.0 * refunds / NULLIF(total_charges, 0), 2) AS refund_rate_pct,

```

```

    total_charged / 100.0 AS gross_usd,
    refund_amount / 100.0 AS refunded_usd
FROM merchant_stats
WHERE refunds::numeric / NULLIF(total_charges, 0) > 0.05 -- > 5% refund rate
ORDER BY refund_rate_pct DESC;

```

Q5. Stripe Connect: calculate platform fee revenue by marketplace.

```

SELECT
    p.platform_id,
    p.name,
    COUNT(c.charge_id) AS transactions,
    SUM(c.amount) / 100.0 AS gross_volume_usd,
    SUM(c.application_fee) / 100.0 AS platform_fees_usd,
    ROUND(100.0 * SUM(c.application_fee) / NULLIF(SUM(c.amount), 0), 4) AS
effective_take_rate
FROM charges c
JOIN platforms p ON c.platform_id = p.platform_id
WHERE c.status = 'succeeded'
    AND c.created_at >= NOW() - INTERVAL '30 days'
GROUP BY p.platform_id, p.name
ORDER BY gross_volume_usd DESC;

```

Database Design — Stripe (Ledger/Financial)

```

-- Double-entry bookkeeping ledger
CREATE TABLE ledger_entries (
    entry_id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    transaction_id    UUID NOT NULL,
    account_id        VARCHAR(50) NOT NULL,
    amount            BIGINT NOT NULL, -- always in cents, never FLOAT
    currency           CHAR(3) NOT NULL,
    direction         CHAR(1) CHECK (direction IN ('D', 'C')), -- debit/credit
    entry_type        VARCHAR(50),
    description       TEXT,
    created_at        TIMESTAMPTZ DEFAULT NOW(),
    idempotency_key   VARCHAR(255) UNIQUE -- prevent duplicate entries
);

-- Balance verification: sum of all debits = sum of all credits
CREATE VIEW ledger_balance_check AS
SELECT
    SUM(CASE WHEN direction = 'D' THEN amount ELSE -amount END) AS net_balance
FROM ledger_entries;

-- Should always equal 0 in a correct double-entry system

```

Key Topics Stripe Emphasizes

- Idempotency: every payment operation must be safe to retry
- Exactly-once semantics for financial transactions

- Strong consistency requirements (no eventual consistency for money)
- NUMERIC precision for currency (never FLOAT/REAL for money)
- Audit trails: every state change logged
- Webhook delivery: at-least-once with deduplication

Salary Bands (USD)

Level	Base	RSU/Equity	Total Comp
L2	\$155K-\$185K	\$120K-\$200K	\$210K-\$300K
L3	\$185K-\$225K	\$200K-\$360K	\$300K-\$450K
L4	\$225K-\$280K	\$360K-\$600K	\$450K-\$650K

DOORDASH

Company Profile

- **Mission:** Grow and empower local economies
- Culture: Operator-first, data-driven, high ownership
- Real-time logistics at scale: 35M+ monthly active users, millions of orders/day
- DB Stack: PostgreSQL (primary OLTP), Amazon Redshift (DWH), Kafka, Redis, Elasticsearch
- Notable: migrated from MongoDB to PostgreSQL — relevant topic in interviews

SQL Questions — DoorDash-Specific

Q1. Calculate Dasher (driver) utilization rate by hour.

```
SELECT
    EXTRACT(HOUR FROM o.created_at) AS hour,
    COUNT(DISTINCT o.dasher_id) AS active_dashers,
    COUNT(o.order_id) AS deliveries,
    ROUND(COUNT(o.order_id)::numeric / NULLIF(COUNT(DISTINCT o.dasher_id), 0), 2) AS
deliveries_per_dasher
FROM orders o
WHERE o.status = 'delivered'
    AND o.created_at >= NOW() - INTERVAL '7 days'
GROUP BY hour
ORDER BY hour;
```

Q2. Find restaurants with consistently high cancellation rates.

```
SELECT
    restaurant_id,
    COUNT(*) AS total_orders,
    COUNT(*) FILTER (WHERE status = 'cancelled') AS cancellations,
    ROUND(100.0 * COUNT(*) FILTER (WHERE status = 'cancelled') / COUNT(*), 2) AS
cancel_rate_pct
FROM orders
WHERE created_at >= NOW() - INTERVAL '30 days'
GROUP BY restaurant_id
```

```
HAVING COUNT(*) >= 50 -- minimum volume
AND 100.0 * COUNT(*) FILTER (WHERE status = 'cancelled') / COUNT(*) > 10
ORDER BY cancel_rate_pct DESC;
```

Q3. Delivery time analysis — find orders that took > 2x the estimated time.

```
SELECT
  order_id,
  restaurant_id,
  dasher_id,
  estimated_delivery_min,
  EXTRACT(EPOCH FROM (actual_delivered_at - placed_at)) / 60 AS
actual_delivery_min,
  ROUND(
    (EXTRACT(EPOCH FROM (actual_delivered_at - placed_at)) / 60) /
    NULLIF(estimated_delivery_min, 0), 2
  ) AS actual_vs_estimated_ratio
FROM orders
WHERE status = 'delivered'
AND actual_delivered_at IS NOT NULL
AND EXTRACT(EPOCH FROM (actual_delivered_at - placed_at)) / 60 >
estimated_delivery_min * 2;
```

Q4. Market penetration: restaurants on DoorDash vs. total restaurants per city.

```
SELECT
  city,
  COUNT(DISTINCT r.restaurant_id) AS on_platform,
  MAX(city_total.total) AS city_total_restaurants,
  ROUND(100.0 * COUNT(DISTINCT r.restaurant_id) / MAX(city_total.total), 2) AS
penetration_pct
FROM restaurants r
JOIN (SELECT city, total FROM city_restaurant_counts) city_total USING (city)
WHERE r.is_active = TRUE
GROUP BY city
ORDER BY penetration_pct DESC;
```

Q5. Cohort analysis: Dasher retention by activation month.

```
WITH cohorts AS (
  SELECT dasher_id, DATE_TRUNC('month', first_delivery_date) AS cohort_month
  FROM (SELECT dasher_id, MIN(delivered_at) AS first_delivery_date FROM deliveries
  GROUP BY dasher_id) t
),
activity AS (
  SELECT DISTINCT dasher_id, DATE_TRUNC('month', delivered_at) AS active_month
  FROM deliveries
)
SELECT
  c.cohort_month,
```



```

COUNT(DISTINCT c.dasher_id) AS cohort_size,
EXTRACT(MONTHS FROM AGE(a.active_month, c.cohort_month)) AS month_number,
COUNT(DISTINCT a.dasher_id) AS retained,
ROUND(100.0 * COUNT(DISTINCT a.dasher_id) / COUNT(DISTINCT c.dasher_id), 1) AS
retention_pct
FROM cohorts c
LEFT JOIN activity a ON c.dasher_id = a.dasher_id
GROUP BY c.cohort_month, month_number
ORDER BY cohort_month, month_number;

```

Key Topics DoorDash Emphasizes

- Real-time logistics and ETA estimation
- Marketplace economics (3-sided: customers, restaurants, Dashers)
- Geographic density and zone management
- Restaurant partner analytics (performance dashboards)
- Fraud: account takeover, promo abuse, fake orders

Salary Bands (USD)

Level	Base	RSU (4yr)	Total Comp
SWE II	\$155K-\$185K	\$120K-\$200K	\$200K-\$290K
SWE III	\$185K-\$225K	\$200K-\$350K	\$290K-\$400K
SWE IV (Senior)	\$225K-\$275K	\$350K-\$600K	\$400K-\$550K

SNOWFLAKE

Company Profile

- **Mission:** Mobilizing the world's data
- B2B SaaS data cloud — they ARE the database product
- Engineering culture: extreme performance focus, cloud-native architecture
- Stack: Cloud-native (multi-cloud: AWS, Azure, GCP), columnar storage, virtual warehouses
- Interview focus: SQL optimization, data modeling, Snowflake-specific features

SQL Questions — Snowflake-Specific

Q1. Optimize a query using Snowflake's time travel.

```

-- Query data as it was 1 hour ago (Snowflake feature)
SELECT * FROM orders AT (OFFSET => -3600);

-- As of a specific timestamp
SELECT * FROM orders AT (TIMESTAMP => '2024-01-15 10:00:00'::timestamp_tz);

-- Compare current vs. historical (audit changes)
SELECT current.*, historical.status AS old_status
FROM orders current
JOIN orders AT (TIMESTAMP => '2024-01-15 09:00:00'::timestamp_tz) historical

```

```
ON current.order_id = historical.order_id
WHERE current.status <> historical.status;
```

Q2. Use Snowflake clustering keys to optimize a partitioned query.

```
-- Define clustering key
ALTER TABLE large_events CLUSTER BY (TO_DATE(event_timestamp), event_type);

-- Query that benefits from clustering
SELECT event_type, COUNT(*), AVG(duration_ms)
FROM large_events
WHERE TO_DATE(event_timestamp) = '2024-01-15'
AND event_type = 'purchase';
-- Micro-partition pruning reduces data scanned ~100x
```

Q3. Snowflake: use QUALIFY instead of nested query for window function filtering.

```
-- Standard: nested query
SELECT * FROM (
  SELECT *, RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rnk
  FROM products
) WHERE rnk <= 3;

-- Snowflake: QUALIFY (clean syntax)
SELECT *, RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rnk
FROM products
QUALIFY rnk <= 3;
```

Q4. Use Snowflake MATCH_RECOGNIZE for pattern detection.

```
-- Detect upward price trend (3 consecutive increases)
SELECT * FROM price_history
MATCH_RECOGNIZE(
  PARTITION BY product_id
  ORDER BY price_date
  MEASURES
    FIRST(price) AS start_price,
    LAST(price) AS end_price,
    COUNT(*) AS streak_length
  PATTERN (UP{3,})
  DEFINE
    UP AS price > LAG(price)
);
```

Q5. Cost optimization: identify most expensive queries in Snowflake.

```
SELECT
  query_id,
  LEFT(query_text, 200) AS query_snippet,
  total_elapsed_time / 1000 AS elapsed_seconds,
```

```

        bytes_scanned / 1e9 AS gb_scanned,
        partitions_scanned,
        partitions_total,
        ROUND(100.0 * partitions_scanned / NULLIF(partitions_total, 0), 2) AS
pct_partitions_scanned
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time >= DATEADD('day', -7, CURRENT_TIMESTAMP)
      AND execution_status = 'SUCCESS'
ORDER BY total_elapsed_time DESC
LIMIT 20;

```

Key Snowflake Topics

- Virtual warehouses: separate compute from storage
- Micro-partition pruning: how clustering keys work
- Snowpipe: continuous data ingestion
- Zero-copy cloning for dev/test environments
- Data sharing: sharing live data across organizations without copying
- Semi-structured data: VARIANT, OBJECT, ARRAY types, FLATTEN, PARSE_JSON

Salary Bands (USD)

Level	Base	RSU (4yr)	Total Comp
SWE II	\$155K-\$195K	\$150K-\$280K	\$230K-\$380K
Senior SWE	\$195K-\$245K	\$280K-\$500K	\$380K-\$600K
Staff SWE	\$245K-\$310K	\$500K-\$900K	\$600K-\$950K

DATABRICKS

Company Profile

- **Mission:** Make data and AI simple for everyone
- Open source core: Apache Spark, Delta Lake, MLflow (all from Databricks)
- B2B enterprise SaaS — data lakehouse platform
- DB Stack: Delta Lake (ACID lakehouse), Apache Spark SQL, Unity Catalog, Delta Sharing
- Interview focus: Spark SQL, Delta Lake, lakehouse architecture, data engineering patterns

SQL Questions — Databricks-Specific

Q1. Use Delta Lake to merge (upsert) incremental data.

```

-- Delta Lake MERGE syntax (supported in Databricks SQL)
MERGE INTO target_table t
USING incremental_data s
ON t.id = s.id
WHEN MATCHED AND s.operation = 'delete' THEN DELETE
WHEN MATCHED THEN UPDATE SET t.value = s.value, t.updated_at = s.updated_at
WHEN NOT MATCHED THEN INSERT (id, value, created_at, updated_at) VALUES (s.id,
s.value, NOW(), NOW());

```

Q2. Time travel in Delta Lake.

```
-- Query previous version
SELECT * FROM events VERSION AS OF 10;
SELECT * FROM events TIMESTAMP AS OF '2024-01-15';

-- Show table history
DESCRIBE HISTORY events;

-- Restore to previous state
RESTORE TABLE events TO VERSION AS OF 5;
```

Q3. Z-ordering for multi-column query optimization.

```
-- Z-order on columns frequently filtered together
OPTIMIZE events
ZORDER BY (user_id, event_date);
-- Skips files that don't match both filters efficiently
```

Q4. Use Databricks autoloader for incremental ingestion.

```
-- Python API for reference
-- SQL equivalent: COPY INTO
COPY INTO bronze_events
FROM 's3://bucket/events/'
FILEFORMAT = JSON
COPY_OPTIONS ('mergeSchema' = 'true')
PATTERN = '*/*.json';
```

Q5. Window functions in Spark SQL (identical to PostgreSQL).

```
SELECT
  user_id,
  event_date,
  event_count,
  SUM(event_count) OVER (PARTITION BY user_id ORDER BY event_date
                        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS rolling_7d
FROM daily_events;
```

Key Databricks Topics

- Medallion architecture: Bronze (raw) → Silver (cleaned) → Gold (aggregated)
- Delta Lake ACID: how Delta achieves ACID on object storage
- Unity Catalog: data governance, lineage, fine-grained access control
- Photon engine: vectorized query execution
- Auto-optimize and auto-compaction

Salary Bands (USD)

Level	Base	RSU (4yr)	Total Comp
-------	------	-----------	------------

SWE II	\$165K–\$200K	\$160K–\$300K	\$250K–\$400K
Senior SWE	\$200K–\$260K	\$300K–\$550K	\$400K–\$650K
Staff SWE	\$260K–\$330K	\$550K–\$1M	\$650K–\$950K

LINKEDIN

Company Profile

- **Mission:** Connect the world's professionals
- Microsoft subsidiary, but largely independent engineering culture
- Data-rich platform: professional profiles, connections, jobs, skills
- DB Stack: Espresso (distributed document store), Voldemort (key-value), Pinot (real-time OLAP), Kafka (Samza), Hadoop/Spark, MySQL (some services)
- PostgreSQL: used for internal tools, analytics

SQL Questions — LinkedIn-Specific

Q1. Find the shortest professional path between two people (degrees of separation).

```
WITH RECURSIVE connection_path AS (
    SELECT member_id AS start, connected_id AS current, 1 AS degree,
    ARRAY[member_id, connected_id] AS path
    FROM connections WHERE member_id = :person_a

    UNION ALL

    SELECT cp.start, c.connected_id, cp.degree + 1, cp.path || c.connected_id
    FROM connection_path cp
    JOIN connections c ON cp.current = c.member_id
    WHERE c.connected_id <> ALL(cp.path) AND cp.degree < 3
)
SELECT degree, path FROM connection_path WHERE current = :person_b ORDER BY degree
LIMIT 1;
```

Q2. Job recommendation: find open positions matching a member's top skills.

```
SELECT
    j.job_id,
    j.title,
    j.company_name,
    COUNT(DISTINCT ms.skill) AS matching_skills,
    ARRAY_AGG(DISTINCT ms.skill) AS matched_skill_list
FROM jobs j
JOIN job_required_skills jrs ON j.job_id = jrs.job_id
JOIN member_skills ms ON jrs.skill = ms.skill AND ms.member_id = :member_id
WHERE j.status = 'open'
    AND j.location_type IN ('remote', 'hybrid')
GROUP BY j.job_id, j.title, j.company_name
```

```
ORDER BY matching_skills DESC
LIMIT 20;
```

Q3. Skills gap analysis: what skills are missing for a target job title.

```
SELECT
    jrs.skill,
    COUNT(DISTINCT j.job_id) AS jobs_requiring_skill,
    CASE WHEN ms.skill IS NULL THEN 'MISSING' ELSE 'HAVE' END AS skill_status
FROM jobs j
JOIN job_required_skills jrs ON j.job_id = jrs.job_id
LEFT JOIN member_skills ms ON jrs.skill = ms.skill AND ms.member_id = :member_id
WHERE j.title ILIKE '%data engineer%'
    AND j.status = 'open'
GROUP BY jrs.skill, ms.skill
ORDER BY jobs_requiring_skill DESC;
```

Q4. Calculate member engagement score.

```
SELECT
    member_id,
    COALESCE(post_count, 0) * 5 +
    COALESCE(comment_count, 0) * 3 +
    COALESCE(reaction_count, 0) * 1 +
    COALESCE(connection_count, 0) * 2 AS engagement_score
FROM members m
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS post_count FROM posts WHERE author_id = m.member_id
    AND created_at >= NOW() - INTERVAL '30 days') posts ON TRUE
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS comment_count FROM comments WHERE author_id = m.member_id
    AND created_at >= NOW() - INTERVAL '30 days') comments ON TRUE
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS reaction_count FROM reactions WHERE member_id = m.member_id
    AND created_at >= NOW() - INTERVAL '30 days') reactions ON TRUE
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS connection_count FROM connections WHERE member_id =
m.member_id) conns ON TRUE
ORDER BY engagement_score DESC;
```

Q5. Trending skills: fastest growing skill endorsements YoY.

```
WITH skill_growth AS (
    SELECT
        skill,
        COUNT(*) FILTER (WHERE endorsed_at >= NOW() - INTERVAL '365 days') AS
current_year,
        COUNT(*) FILTER (WHERE endorsed_at BETWEEN NOW() - INTERVAL '730 days' AND
NOW() - INTERVAL '365 days') AS prior_year
    FROM skill_endorsements
```

```

        GROUP BY skill
    )
SELECT
    skill,
    current_year,
    prior_year,
    ROUND(100.0 * (current_year - prior_year) / NULLIF(prior_year, 0), 2) AS
yoy_growth_pct
FROM skill_growth
WHERE prior_year > 1000 -- minimum base
ORDER BY yoy_growth_pct DESC
LIMIT 20;

```

Salary Bands (USD)

Level	Base	RSU (4yr)	Total Comp
SWE II	\$150K-\$185K	\$100K-\$200K	\$220K-\$340K
Senior SWE	\$185K-\$235K	\$200K-\$380K	\$340K-\$490K
Staff SWE	\$235K-\$295K	\$380K-\$700K	\$490K-\$700K

ATLASSIAN

Company Profile

- **Mission:** Unleash the potential of every team
- Products: Jira, Confluence, Trello, Bitbucket, OpsGenie
- Australian roots, global engineering, distributed-first culture
- DB Stack: PostgreSQL (primary), AWS Aurora, Kafka, OpenSearch, Redis, Spark
- Strong PostgreSQL presence — Jira cloud uses PostgreSQL at scale

SQL Questions — Atlassian-Specific

Q1. Jira: Find all overdue tickets by team.

```

SELECT
    t.team_id,
    t.team_name,
    COUNT(*) AS overdue_tickets,
    AVG(CURRENT_DATE - i.due_date) AS avg_days_overdue
FROM issues i
JOIN teams t ON i.team_id = t.team_id
WHERE i.due_date < CURRENT_DATE
    AND i.status NOT IN ('Done', 'Closed')
GROUP BY t.team_id, t.team_name
ORDER BY overdue_tickets DESC;

```

Q2. Sprint velocity analysis: story points completed per sprint.

```

SELECT
    s.team_id,
    s.sprint_number,
    s.sprint_name,
    SUM(i.story_points) FILTER (WHERE i.status = 'Done') AS completed_points,
    SUM(i.story_points) AS total_committed_points,
    ROUND(100.0 * SUM(i.story_points) FILTER (WHERE i.status = 'Done')
          / NULLIF(SUM(i.story_points), 0), 2) AS completion_rate
FROM sprints s
JOIN sprint_issues si ON s.sprint_id = si.sprint_id
JOIN issues i ON si.issue_id = i.issue_id
WHERE s.status = 'completed'
GROUP BY s.team_id, s.sprint_number, s.sprint_name
ORDER BY s.team_id, s.sprint_number;

```

Q3. Cycle time analysis: time from "In Progress" to "Done".

```

WITH status_changes AS (
    SELECT
        issue_id,
        status,
        changed_at,
        LEAD(changed_at) OVER (PARTITION BY issue_id ORDER BY changed_at) AS
next_change_at
    FROM issue_status_history
),
cycle_time AS (
    SELECT
        issue_id,
        SUM(EXTRACT(EPOCH FROM (COALESCE(next_change_at, NOW()) - changed_at)) /
3600) AS hours_in_progress
    FROM status_changes
    WHERE status = 'In Progress'
    GROUP BY issue_id
)
SELECT
    i.project_key,
    i.issue_type,
    ROUND(AVG(ct.hours_in_progress) / 24, 1) AS avg_cycle_time_days,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ct.hours_in_progress / 24) AS
median_days,
    PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY ct.hours_in_progress / 24) AS
p90_days
FROM cycle_time ct
JOIN issues i ON ct.issue_id = i.issue_id
GROUP BY i.project_key, i.issue_type;

```

Q4. Confluence: find pages that haven't been updated in 6+ months (knowledge debt).


```
SELECT
    space_key,
    page_id,
    title,
    last_updated_at,
    view_count_last_30d,
    CURRENT_DATE - last_updated_at::date AS days_stale
FROM confluence_pages
WHERE last_updated_at < NOW() - INTERVAL '180 days'
    AND status = 'published'
    AND view_count_last_30d > 50 -- still being accessed but stale
ORDER BY days_stale DESC;
```

Q5. Bitbucket: code review turnaround time analysis.

```
SELECT
    pr.repo_id,
    pr.author_id,
    ROUND(AVG(EXTRACT(EPOCH FROM (pr.first_review_at - pr.created_at)) / 3600), 1)
AS avg_hours_to_first_review,
    ROUND(AVG(EXTRACT(EPOCH FROM (pr.merged_at - pr.created_at)) / 3600), 1) AS
avg_hours_to_merge,
    COUNT(*) AS pr_count
FROM pull_requests pr
WHERE pr.status = 'merged'
    AND pr.created_at >= NOW() - INTERVAL '90 days'
GROUP BY pr.repo_id, pr.author_id
ORDER BY avg_hours_to_first_review DESC;
```

Salary Bands (USD)

Level	Base	RSU (4yr)	Total Comp
SWE II	\$140K-\$175K	\$80K-\$150K	\$190K-\$290K
Senior SWE	\$175K-\$225K	\$150K-\$280K	\$290K-\$420K
Staff SWE	\$225K-\$285K	\$280K-\$500K	\$420K-\$600K

Sydney/Melbourne rates ~70% of US. Amsterdam ~80%.

Cross-Company Comparison Table

Company	Primary DB	PostgreSQL Role	Key SQL Focus	Top Interview Topics
Airbnb	MySQL + Presto	Internal tools	Marketplace analytics	Occupancy, pricing, host metrics
Stripe	PostgreSQL	CORE system	Financial precision	Idempotency, ledger, fraud

DoorDash	PostgreSQL + Redshift	CORE + analytics	Logistics analytics	Delivery time, Dasher utilization
Snowflake	Snowflake (their product)	N/A (IS the product)	Snowflake-specific SQL	Virtual warehouses, clustering, time travel
Databricks	Delta Lake (Spark SQL)	N/A (different paradigm)	Spark SQL, Delta Lake	Medallion architecture, Z-order
LinkedIn	Espresso + Pinot	Internal analytics	Graph queries, skills	Professional network analysis
Atlassian	PostgreSQL + Aurora	CORE system	Project analytics	Sprint velocity, cycle time

Universal Preparation Tips for All Companies

1. Research the Stack

Before any interview:

- Check the company's engineering blog (all above companies have excellent ones)
- Know their primary database and why they chose it
- Understand their data scale (orders of magnitude)
- Know one or two specific blog posts about their DB architecture

2. Domain-Adapted SQL

Every company wants SQL adapted to their domain:

- **Marketplace** (Airbnb, DoorDash): supply/demand ratios, cohort retention, funnel analysis
- **Financial** (Stripe): precision arithmetic, idempotency, audit trails
- **Analytics platforms** (Snowflake, Databricks): query optimization, partitioning, semi-structured data
- **Social/Professional** (LinkedIn, Meta): graph queries, network analysis, activity metrics
- **SaaS/Productivity** (Atlassian): SLAs, cycle time, sprint/project analytics

3. Common Hard SQL Patterns All Companies Test

```
-- 1. Gaps and Islands
-- 2. Rolling N-day windows
-- 3. Year-over-year / period comparisons
-- 4. Cohort retention tables
-- 5. Funnel analysis (step conversion rates)
-- 6. Percentile calculations (P50, P95, P99)
-- 7. Recursive hierarchies (org charts, categories)
-- 8. Market basket / co-occurrence analysis
-- 9. Session analysis (gap > N minutes = new session)
-- 10. Ranking within groups (top N per category)
```

4. System Design Framework

For any DB-focused system design at these companies:

1. **Clarify scale:** rows, queries/sec, read vs. write ratio
2. **Define entities:** core tables, relationships
3. **Index strategy:** which columns, which types
4. **Partitioning:** by what key, what interval
5. **Caching:** what lives in Redis, TTL, invalidation strategy
6. **Replication:** read replicas, failover requirements
7. **Analytics:** how OLAP queries are served (materialized views, separate DWH)
8. **Monitoring:** which metrics matter, what triggers alerts

5. Behavioral Themes Per Company

Company	Core Behavioral Theme
Airbnb	Belonging, community impact, creative solutions
Stripe	Technical depth, precision, developer empathy
DoorDash	Operator mindset, data-driven decisions, ownership
Snowflake	Scale thinking, customer obsession, technical innovation
Databricks	Open source contribution, collaboration, innovation
LinkedIn	Member value, professional impact, trust
Atlassian	Team collaboration, openness, balance